

HW05 DEMO-2

PROGRAM ASSIGNMENT

資料結構與演算法

迷宮路徑搜尋

DFS & BFS

目標

在隨機生成的迷宮中，用 DFS 和 BFS 找出從起點到終點的路徑
需使用 Linked List 自行實作 Stack (DFS) 與 Queue (BFS)
不可使用 STL 容器 (std::stack, std::queue, std::vector 等)

檔案結構

Maze.cpp	← 迷宮核心 (已有，需補完部分函式)
StudentSolver.cpp	← 主要演算法 (DFS / BFS)
main.cpp	← 主程式 (已提供，不需修改)

PATHNODE

路徑節點

maze.cpp

節點結構

已提供

```
struct PathNode {  
    int r, c;        // 迷宮座標 (列, 行)  
    PathNode* next;  // 指向下一個節點  
}; (包含free list)
```

用途

學生實作

1. Stack 的節點 (DFS 用)
2. Queue 的節點 (BFS 用)
3. 最終路徑的 linked list

MAZE類別

迷宮的核心

maze.cpp

PRIVATE

n grid 總邊長（含邊界）
grid 二維陣列，0 = 通道，1 = 牆
sR, sC 起點座標
eR, eC 終點座標

建構

Maze

自動生成 $(size+2) \times (size+2)$ 的迷宮 (包含邊界)

size 必須是奇數且 ≥ 15 & ≤ 99

seed = 0 表示隨機，其他值可重現同一張迷宮

迷宮四周有一圈邊界牆，起點 S 在上方開口，終點 E 在下方開口

MAZE API

可使用之函式

maze.cpp

PUBLIC

已提供

getN() → 回傳 grid 總邊長
getStartR() → 起點的 row
getStartC() → 起點的 col
getEndR() → 終點的 row
getEndC() → 終點的 col

注意

grid 是 private，無法直接用 grid[r][c] 存取

maze.cpp

S → 起點

+ - | → 邊界

• → 通道

從起點S開始

不能斜著走

不能走上邊界、牆壁

最終走到終點E

The figure shows a 15x15 grid with the following labels:

- Top row: + S - - - - - - - - - - - - - +
- Bottom row: + - - - - - - - - - - - - - E +
- Left column: + (vertical line) +
- Right column: (vertical line) +

The grid contains a pattern of '#' characters. The pattern is roughly as follows (row by row, from top to bottom):

- Row 1:
- Row 2: # # # # # . # . # .
- Row 3: . . . # #
- Row 4: . # . # # # . . # . # . . # .
- Row 5: . # . . . # # .
- Row 6: . # # # # # # . # # # .
- Row 7: . # # . . # . # . . .
- Row 8: . # # . # # # .
- Row 9: . # . . . # # . . . # .
- Row 10: . # . . . # . . . # . # # . # .
- Row 11: # # . # # .
- Row 12: . # # # . # # . . # # . # . # .
- Row 13: . # # . # . . .
- Row 14: . # # . . # . # . # . # # # .
- Row 15:

MAZE類別

展示迷宮(含路徑)

maze.cpp

displayPath()

已寫好，不需修改

左邊顯示原始迷宮，右邊只顯示路徑

路徑上的格子用 * 標示

當你的 DFS/BFS 找到路徑後，main.cpp 會自動呼叫此函式

EXAMPLE

圖示在下一頁

===== DFS =====

DFS: path found!

Maze

S	-	-	-	-	-	-	-	-	-	-	-	-	-	-
.	#	.	.	.	.	.	.	.	.	.	.	.	.	.
.	#	#	.	.	.	#	#	#	#	.	#	#	#	.
.	.	.	.	.	#	.	.	.	.	.	.	.	#	.
#	.	.	#	#	.	.	#	.	.	#	#	.	#	.
.	#	.	.	.	.	.	.	.	.	.	.	.	#	.
.	#	.	#	.	#	#	.	#	#	.	#	#	#	.
.	.	.	.	.	.	.	.	.	.	.	.	.	#	.
.	#	#	.	#	#	.	.	.	#	.	.	#	#	.
.	.	.	.	.	.	.	#	.	.	.	#	.	.	.
.	#	.	#	.	#	#	#	.	#	.	#	.	#	#
.	#	.	.	.	#	.	#	.	.	.	.	.	#	.
.	.	#	#	#	#	.	.	#	#	#	#	.	#	.
.	#	.	.	.	#	.	.	.	.	.	.	.	.	.
.	#	.	#	.	#	.	.	#	.	.	.	.	#	.
.	.	.	#	.	.	.	.	.	.	.	.	.	.	.
-	-	-	-	-	-	-	-	-	-	-	-	-	-	E

Path

S	-	-	-	-	-	-	-	-	-	-	-	-	-	-
*														
*														
*	*													
	*	*												
		*												
		*												
		*	*											
			*			*	*	*						
			*	*	*	*		*						
								*						
								*	*	*	*	*		
												*		
												*		
												*	*	*
-	-	-	-	-	-	-	-	-	-	-	-	-	-	E

[Validate] PASS (steps: 32)

實作部分

TODO 1

StudentSolver.cpp

DFS

位於StudentSolver.cpp，請刪掉重寫(函式名稱勿動)

```
bool solveByDFS(const Maze& maze, PathNode*& path) {  
    int n = maze.getN(); (void)n;    path = nullptr;  
    //TODO 1 : Solve by DFS  
    return false; }
```

邏輯

1. 取得迷宮資訊：n, 起點, 終點
2. 實作輔助陣列：visited[][], parentR[][], parentC[][]
3. 用 PathNode 的 linked list 實作 Stack (LIFO)
4. 從起點開始，每次 pop 一個節點，嘗試上下左右四方向
5. 到達終點 → 用 parent 陣列回溯建立路徑 linked list
6. 釋放所有陣列與剩餘的 stack 節點
7. 找到路徑 → path = 路徑 linked list，回傳 true；無解 → path = nullptr，回傳 false

實作部分

TODO 2

StudentSolver.cpp

BFS

位於StudentSolver.cpp，請刪掉重寫(函式名稱勿動)

```
bool solveByBFS(const Maze& maze, PathNode*& path) {  
    int n = maze.getN(); (void)n; path = nullptr;  
    //TODO 2 : Solve by BFS  
    return false; }
```

邏輯

1. 與 DFS 相同的前置作業（取得資訊、配置陣列）
2. 用 PathNode 的 linked list 實作 Queue（FIFO）
 - enqueue：new PathNode 接在 tail
 - dequeue：取 head 的座標，移除 head
 - 需同時維護 head 和 tail 兩個指標
3. 從起點開始，每次 dequeue 一個節點，嘗試上下左右
4. BFS 找到的路徑保證是最短路徑
5. 其餘（回溯、釋放、回傳）同 DFS

實作部分

TODO 3

maze.cpp

validatePath

位於maze.cpp最後面，請刪掉重寫(函式名稱勿動)

```
bool Maze::validatePath(PathNode* path) const {  
    // TODO 3: 請實作路徑驗證  
    (void)path;  
    return false; }
```

邏輯

1. 檢查 path 是否為 nullptr → 回傳 false
2. 檢查路徑起點是否等於迷宮起點 (sR, sC)
3. 逐步走訪 linked list：每一步必須合法且與前一步相鄰
4. 檢查路徑終點是否等於迷宮終點 (eR, eC)
5. 全部通過 → 回傳 true

提示：需要存取 grid 的值來判斷格子是否為通道

程式執行 手動測試

main.cpp

run

已寫好，不需修改

執行流程：

1. 輸入迷宮大小 n (奇數，15~99)
2. 輸入 random seed (0 = 隨機)
3. 建立迷宮 `Maze maze(n, seed)`
4. 呼叫你寫的 `solveByDFS` → 顯示結果
5. 呼叫你寫的 `solveByBFS` → 顯示結果

找到路徑 → 並排顯示迷宮與路徑 + `validatePath` 驗證

沒找到 → 只顯示迷宮

```
Enter maze size (odd number >= 15): 15
Enter random seed (0 = random): 0
```

程式執行 自動測試

main.cpp

autoTest

已寫好，不需修改

執行檔 + "--test"

內建 6 組測試案例 (n = 15, 17, 19, 20, 69, 99) (可改)

每組測試：

1. 生成迷宮並顯示
2. 跑 DFS → validatePath 驗證 → PASS / FAIL
3. 跑 BFS → validatePath 驗證 → PASS / FAIL
4. 比較 BFS 路徑長度 ≤ DFS 路徑長度

最後統計 PASS / FAIL 數量

- PASS = 你的路徑通過 validatePath 驗證（起點正確、終點正確、每步合法且相鄰）
- FAIL = 路徑有誤或未找到路徑，代表你的 DFS/BFS 或 validatePath 實作有 bug

總結

TODO

檔案	函式	說明
StudentSolver.cpp	solveByDFS()	DFS 搜尋
StudentSolver.cpp	solveByBFS()	BFS 搜尋
Maze.cpp	validatePath()	路徑驗證

注意事項

main.cpp 僅限修改測試案例，其餘部分禁止更動。

Maze.cpp 嚴禁修改原程式碼，但可視需求新增自訂函式。

作業繳交須知

繳交格式

壓縮檔：HW05_學號_姓名.zip

執行檔：HW05_學號.exe

原始檔案：

main.cpp / maze.cpp /

StudentSolver.cpp

(原始檔案請勿更名)

若以上一項沒符合，一律-10

DEMO時間

05/11(一)~05/15(五)

評分標準

請熟悉所有程式碼，包含提供的部分以及你自己寫的三個 TODO

請勿更改任何TODO以外之程式碼，更動者0分計算